

Name : Hemant Parmar

Prn : 202501070129

Section : ET2

Batch : E21

Subject : ESSENTIAL OF DATA SCIENCE (EDS)

PRACTICAL 1

1.1 Practical Lab Assignment

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

is the velocity of the object (in meters per second).

Input Format:

A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
m = float(input())  
v = float(input())  
p = m * v  
print(f'{p:.2f}kgm/s')
```

OUTPUT:

The screenshot displays a code execution interface with a dark theme. At the top, performance metrics are shown: Average time 0.008 s (7.75 ms) and Maximum time 0.010 s (10.00 ms). Test results indicate that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. Below this, two test cases are detailed. Test case 1 shows expected output 5.0, 10.0, and 50.00kgm/s, which matches the actual output. Test case 2 shows expected output 2.3, 4.8, and 11.04kgm/s, which also matches the actual output. Each test case entry includes a green checkmark, a 'DEBUG' button, and a menu icon.

Metric	Value
Average time	0.008 s (7.75 ms)
Maximum time	0.010 s (10.00 ms)
Shown test cases	2 out of 2 passed
Hidden test cases	2 out of 2 passed

Test Case	Expected Output	Actual Output
Test case 1	5.0 10.0 50.00kgm/s	5.0 10.0 50.00kgm/s
Test case 2	2.3 4.8 11.04kgm/s	2.3 4.8 11.04kgm/s

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

$1 \leq \leq 999$

Input Format:

The input consists of a single integer .

Output Format:

If is a single-digit number, print its square.

If is a two-digit number, print its square root (rounded to two decimal places).

If is a three-digit number, print its cube root (rounded to two decimal places).

Else print "Invalid".

Code:

```
n = int(input())

if n >= 0 and n < 10 :

    print(n**2)

elif n >= 10 and n < 100:

    r = n**(1/2)

    print(f'{r:.2f}')

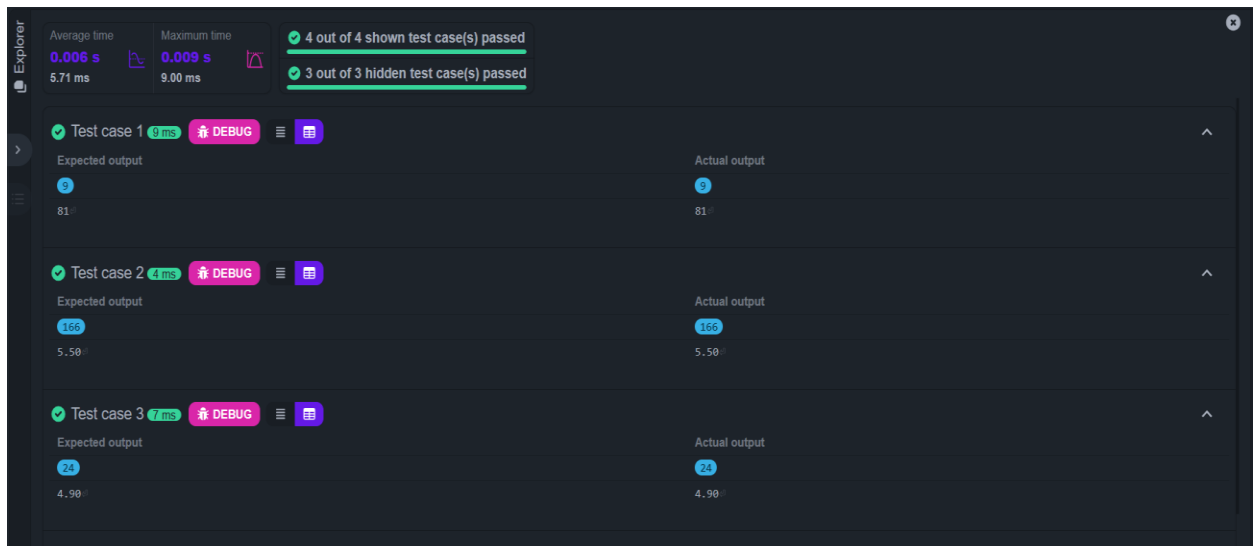
elif n >= 100 and n < 1000:

    q = n**(1/3)

    print(f'{q:.2f}')

else: print("Invalid")
```

Output:



1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

The first line contains the first date in the format YYYY-MM-DD.

The second line contains the second date in the format YYYY-MM-DD .

Output Format:

An integer representing the number of days between the given dates.

Note:

The first date should always be considered earlier than the second date.

Code:

```
from datetime import date
```

```
from datetime import datetime
```

```
a = input().strip()
```

```
b = input().strip()
```

```
d1 = datetime.strptime(a,"%Y-%m-%d")
```

```
d2 = datetime.strptime(b,"%Y-%m-%d")
```

```
difference = d2 - d1
```

```
print(difference.days)
```

Output:

The screenshot shows a test runner interface with a dark theme. At the top, it displays performance metrics: Average time 0.010 s (9.75 ms) and Maximum time 0.020 s (20.00 ms). It also shows the test results: 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. Below this, there are two test case details. Test case 1, which took 20 ms, shows an expected output of 2014-07-02 and 2014-11-02, and an actual output of 2014-07-02 and 2014-11-02. Test case 2, which took 7 ms, shows an expected output of 2014-07-02 and 2014-07-11, and an actual output of 2014-07-02 and 2014-07-11. Both test cases are marked as passed with green checkmarks.

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

The input is a single integer.

Output Format:

The output prints a single integer, which is the reversed number.

Code:

```
n = int(input())  
rev_n = int(str(n)[::-1])  
print(rev_n)
```

OUTPUT:

The screenshot shows a code execution interface for a program named 'reverseN...'. The top status bar indicates '2 out of 2 shown test case(s) passed' and '3 out of 3 hidden test case(s) passed'. Below this, two test cases are detailed:

Test Case	Expected output	Actual output
Test case 1	5367 7635	5367 7635
Test case 2	0132 231	0132 231

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

Print the multiplication table for the given number in the format:

`<number> x <multiplier> = <result>`

Note:

The character "x" is used in the output to represent multiplication.

Refer to the visible test-cases for more clarity.

Code:

```
n = int(input())  
for i in range(1,11):  
    print(n,"x",i,"=",i*n)
```

OUTPUT:

The screenshot displays a code execution interface with the following details:

- Performance Metrics:**
 - Average time: 0.009 s (8.60 ms)
 - Maximum time: 0.019 s (19.00 ms)
- Test Results:**
 - 2 out of 2 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1 Details:**
 - Expected output:**

```
8  
8 x 1 = 8  
8 x 2 = 16  
8 x 3 = 24  
8 x 4 = 32  
8 x 5 = 40  
8 x 6 = 48  
8 x 7 = 56  
8 x 8 = 64  
8 x 9 = 72  
8 x 10 = 80
```
 - Actual output:**

```
8  
8 x 1 = 8  
8 x 2 = 16  
8 x 3 = 24  
8 x 4 = 32  
8 x 5 = 40  
8 x 6 = 48  
8 x 7 = 56  
8 x 8 = 64  
8 x 9 = 72  
8 x 10 = 80
```
- Test Case 2:** Also passed, with expected and actual outputs matching.

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

If the aggregate percentage is greater than 75, the grade is Distinction.

If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.

If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

The first input will be an integer , the number of courses.

The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

Print the aggregate percentage (formatted to two decimal places).

Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

"Fail".

Note:

Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Code:

```
n = int(input())
```

```
marks = list(map(int,input().split()))
```

```
failed = False
```

```
for m in marks:
```

```
if (m<40):
```



```
failed = True  
break  
if failed:  
print("Fail")  
else:  
total = sum(marks)  
agg = total / n  
print("Aggregate Percentage: "{agg:.2f}")  
  
if agg > 75:  
print("Grade: Distinction")  
elif agg >= 60:  
print("Grade: First Division")  
elif agg >= 50:  
print("Grade: Second Division")  
elif agg >= 40:  
print("Grade: Third Division")
```

Output:

Explorer

Average time: 0.009 s (9.00 ms) Maximum time: 0.024 s (24.00 ms)

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1 (24 ms) [DEBUG]

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75%	Aggregate Percentage: 71.75%
Grade: First Division	Grade: First Division

Test case 2 (8 ms)

Test case 3 (9 ms)

Test case 4 (10 ms)

Test case 5 (7 ms)

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first `n` terms of the Fibonacci series.

Input Format:

A single integer `n` representing the number of terms to generate.

Output Format:

A single line containing the first `n` terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n):
```

```

if n == 1:

    return 0

elif n == 2:

    return 1

else:

    return fibonacci(n-1)+fibonacci(n-2)


n = int(input())

for i in range(1, n + 1):

    print(fibonacci(i), end=" ")

```

OUTPUT:

The screenshot displays a code editor with a Python program for calculating the Fibonacci series. The program takes an integer n as input and prints the first n terms of the Fibonacci sequence, separated by spaces. The code is as follows:

```

def fibonacci(n):
    if n == 1:
        return 0
    elif n == 2:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

n = int(input())
for i in range(1, n + 1):
    print(fibonacci(i), end=" ")

```

The interface also shows the input format (a single integer n), the output format (a single line of the first n terms), and sample test cases. Test case 1 expects the output "0 1 1 2 3" for input 5, and Test case 2 expects the output "0 1 1 2 3 5 8 13 21 34 55 89 144 233 377" for input 15. The test results on the right indicate that both test cases passed, with 2 out of 2 shown and hidden test cases passing. Performance metrics show an average time of 0.018 s and a maximum time of 0.028 s.

1.2.3. Pattern – 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format

The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

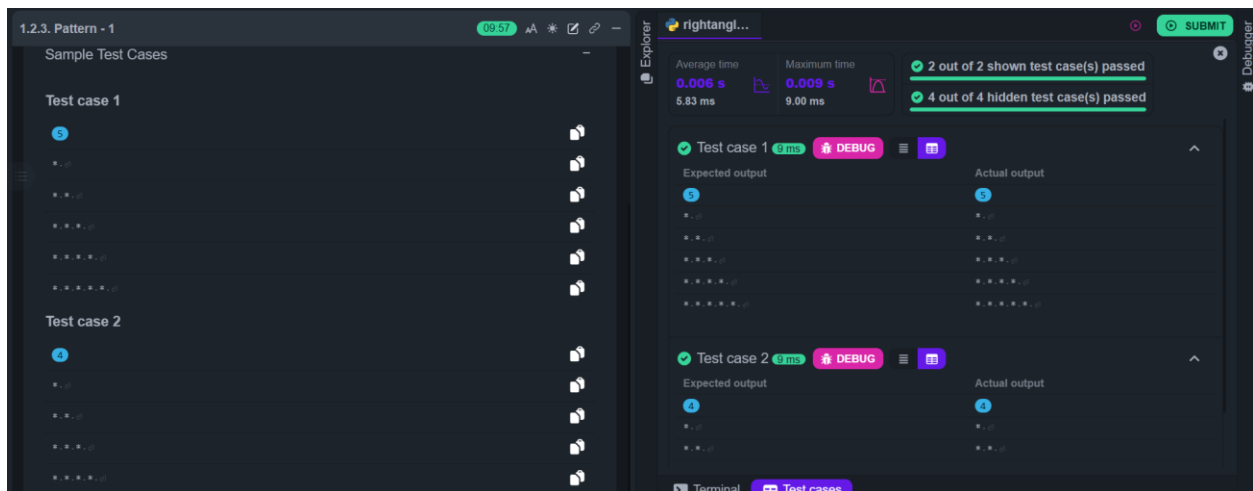
Code:

```
n = int(input())

for i in range(1,n+1):

    print('* ' * i)
```

OUTPUT:



1.2.4. Pattern – 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

Refer to the displayed test cases for the sample pattern.

Code:

```
n = int(input())

for i in range(1, n+1):

    for j in range(1,i+1):

        print(j,end=" ")

    print()
```

OUTPUT:

The screenshot displays a code editor interface with a dark theme. On the left, a sidebar titled "Sample Test Cases" lists two test cases. Test case 1 has an input of 5 and an expected output of a 5x5 number pattern. Test case 2 has an input of 4 and an expected output of a 4x4 number pattern. The main editor area shows the code for "numberP...". On the right, a panel displays the execution results, indicating that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The average time is 0.004 s (4.00 ms) and the maximum time is 0.006 s (6.00 ms). The panel also shows the expected and actual outputs for both test cases, which match perfectly.

Test Case	Input	Expected Output	Actual Output
Test case 1	5	1 1 2 1 2 3 1 2 3 4 1 2 3 4 5	1 1 2 1 2 3 1 2 3 4 1 2 3 4 5
Test case 2	4	1 1 2 1 2 3 1 2 3 4	1 1 2 1 2 3 1 2 3 4

